TECNICATURA UNIVERSITARIA EN DESARROLLO WEB · FCAD · UNER

Unidad 2
Depuración y rendimiento

Desarrollo para Móviles · 2026 — Segundo cuatrimestre

Prof. Titular: Lic. Franco Ruis Dias · JTP: Lic. Germán Cueva

React Native 0.86 · Expo SDK 57

UNIDAD 2

Qué vamos a ver

Objetivos

- Dejar de depurar con `console.log` a ciegas.
- Leer lo que pasa dentro de la app mientras corre.
- Encontrar por qué algo va lento, con datos.

Temas

- El menú de desarrollo
- **React Native DevTools**
- Frames, hilos y el Perf Monitor
- Listas largas

> Todo lo de hoy es **gratis**: viene con el proyecto y no hay que instalar nada. Es la clase con mejor relación entre lo que cuesta y lo que ahorra.

UNIDAD 2 · PARTE 1

El menú de desarrollo

La puerta de entrada a todo lo demás.

Nada de esto existe en producción

Build de debug

Menú de desarrollo, LogBox, DevTools, Fast Refresh. La app hace mucho trabajo extra para poder avisarte de las cosas.

Build de release

Todo eso **está deshabilitado**. El código va optimizado y minificado.

De acá sale la regla más importante de la segunda mitad de la clase: **el rendimiento se mide en release, nunca en debug.** En debug la app es lenta a propósito, y medir ahí lleva a optimizar cosas que no son el problema.

Cómo se abre

Desde la terminal

Con el servidor de desarrollo corriendo, apretá `m`. Es la forma más cómoda y la que menos falla.

Y `j` abre directamente las DevTools.

Desde el dispositivo

- **Android:** sacudir, o `Ctrl` / `Cmd` + `M` en el emulador.
- **iOS:** sacudir, tres dedos en la pantalla, o `Ctrl` + `Cmd` + `Z` en el simulador.

```
adb shell input keyevent 82    # Android, cuando sacudir no funciona
```

Qué trae el menú

Open DevTools
Lo importante. Es la Parte 2 entera.

Toggle performance monitor
El overlay con FPS, memoria y cantidad de vistas. Parte 3.

Toggle element inspector
Tocás un elemento en pantalla y te muestra qué componente es, con sus estilos y su jerarquía.

Fast Refresh · Reload
Fast Refresh está activado por defecto y conserva el estado. Recargar sólo hace falta cuando algo quedó raro.

> En Expo el menú lo proveen **Expo Go** y los development builds. Si armaste una build de release, el menú no está — y eso es lo esperado.

UNIDAD 2 · PARTE 2

React Native DevTools

El depurador que viene incluido.

Qué es, y qué reemplazó

Desde **React Native 0.76**, la experiencia de depuración oficial. Está construida sobre el frontend de Chrome DevTools, así que si venís de web ya la conocés. Funciona en cualquier app que use **Hermes**.

Lo que quedó atrás NO USAR

- **Flipper**
- El depurador de Hermes en Chrome
- `chrome://inspect` — ya no está soportado
- **Remote JS Debugging**, deprecado desde 0.73

Cómo se abre

Apretando `j` en la terminal, o Open DevTools en el menú de desarrollo.

> Casi todos los tutoriales anteriores a 2025 explican Flipper o `chrome://inspect`. Si encontrás uno, cerralo.

Console

Es una terminal conectada a tu app

Podés escribir JavaScript y se ejecuta adentro de la aplicación. Si estás parado en un breakpoint, se ejecuta en ese ámbito: ves y modificás las variables locales.

Trucos que valen

- Filtrar por texto o por nivel cuando hay mucho ruido.
- **Preserve logs** para que sobrevivan a un reload.
- Live Expressions para vigilar un valor sin llenar de logs.

> Con las DevTools abiertas, **LogBox oculta todo salvo los errores fatales**. La consola de DevTools es la fuente de verdad: LogBox tiene filtros propios que pueden esconderte cosas.

Breakpoints

```
function calcularTotal(items: Item[]) {
  debugger;                    // para la ejecución acá
  return items.reduce((a, i) => a + i.precio, 0);
}
```

Dos formas

Hacer clic en el número de línea en la pestaña Sources, o escribir `debugger;` en el editor — Fast Refresh lo lleva al dispositivo al instante.

Cuando está parado

Mirá los paneles de la derecha: ámbito actual, pila de llamadas y expresiones vigiladas. Ahí está la respuesta, no en el `console.log` que ibas a escribir.

> Hay **breakpoints condicionales** y logpoints: paran sólo si se cumple una condición, o imprimen sin parar. Sirven para el caso «falla recién en el ítem 400».

Network

Qué muestra

Cada request con sus tiempos, cabeceras enviadas y recibidas, y una vista previa de la respuesta. La pestaña Initiator te dice desde qué línea de tu código salió.

La particularidad de Expo

Expo muestra su propio panel **Expo Network**, que cubre más orígenes de request. Requiere `expo-dev-client` o Expo Go.

> Se registra automáticamente mientras las DevTools estén abiertas. Todavía no hay WebSockets, ni simulación de red lenta, ni respuestas simuladas.

Components

El árbol, en vivo

Ves el árbol de componentes de React con sus props y su estado. Y podés **editarlos en caliente** para probar sin tocar el código.

El botón «Select element» arriba a la izquierda: lo apretás, tocás algo en el teléfono y te lleva al componente.

Es React, no la pantalla

El element inspector del menú te muestra la vista nativa. Este panel te muestra **tu componente**. Son dos árboles distintos y conviene no confundirlos.

> Los componentes optimizados por **React Compiler** aparecen con una insignia «Memo ✨».

El truco que más rinde

Resaltar re-renderizados. En el engranaje de View Settings, activá «Highlight updates when components render».

Qué vas a ver

Un borde de color alrededor de cada componente cada vez que se vuelve a renderizar. Escribís una letra en un campo de texto y se te ilumina media pantalla.

Por qué importa

Los re-renderizados de más son la causa número uno de lentitud en una app React. Y son invisibles hasta que los pintás.

> Es también la forma honesta de decidir si hace falta `useMemo`: primero mirás, después optimizás. Al revés, se agrega complejidad sin evidencia.

Profiler y Memory

React Profiler

Grabás mientras usás la app y te muestra, en un gráfico de llamas, cuánto tardó cada componente en renderizar y cuántas veces lo hizo.

Sirve para responder «¿qué componente se está comiendo el tiempo?».

Memory

Instantáneas del montón de memoria de JavaScript. Comparar dos instantáneas separadas en el tiempo es cómo se detecta una fuga.

Típicamente: el efecto sin limpieza de la clase de ciclo de vida.

> Los perfiles todavía **no se simbolizan con source maps** y sólo se pueden tomar en builds de debug. Sirven para comparar, no para dar números absolutos.

UNIDAD 2 · PARTE 3

Rendimiento

Qué significa «va lento», y cómo se mide.

16,67 milisegundos

Los dispositivos muestran **al menos 60 cuadros por segundo**. Eso deja 16,67 ms para preparar cada cuadro. Si no llegás, se pierde el cuadro y la interfaz se siente trabada.

60	16,67	12
cuadros por segundo	milisegundos por cuadro	cuadros perdidos si algo tarda 200 ms

> «Va lento» casi nunca es un problema general de la app: es **una operación concreta** que no entra en su presupuesto de tiempo. El trabajo es encontrar cuál.

Los dos hilos

Hilo de JavaScript

Tu lógica, los eventos táctiles, las llamadas a la API, el renderizado de React. Si se traba, la app deja de responder a los toques.

Hilo de UI (principal)

El dibujado nativo. Puede seguir andando aunque el de JavaScript esté ocupado.

> Por eso un `ScrollView` sigue desplazándose fluido con el hilo de JS trabado, y por eso las transiciones del **native stack** se sienten mejores que las de un stack en JavaScript: corren del lado nativo. Es el mismo tema de la arquitectura de ejecución de la clase anterior.

UNIDAD 2 · PARTE 3 · RENDIMIENTO

Perf Monitor

Desde el menú de desarrollo: Toggle performance monitor. Un overlay chiquito sobre la app.

RAM
Memoria que ocupa el proceso.

JSC / heap
Memoria de JavaScript. Si sube y nunca baja, hay una fuga.

Views
Cantidad de vistas en pantalla y en el componente.

UI · JS
Los dos FPS, por separado. Es el dato principal.

Es **una guía, no una medición**: corre dentro de la app y por lo tanto la afecta. Para números serios están las herramientas de Android Studio y Xcode.

Las dos trampas de siempre

Medir en debug

En modo desarrollo el hilo de JavaScript hace mucho trabajo extra para poder darte advertencias útiles. La app es más lenta, y no tiene nada que ver con cómo va a andar publicada.

`console.log` en el código que se publica

Cada llamada cuesta, y en un bucle o en un render cuesta mucho. Incluyendo los que dejaron tus dependencias.

```
npm i babel-plugin-transform-remove-console --save-dev
```

> Ese plugin, configurado para el entorno de producción, borra todas las llamadas a `console.*` de la build de release. Conviene ponerlo aunque no uses ninguna.

Listas largas

`map()` dentro de un `ScrollView`

Renderiza **todos** los elementos, estén o no en pantalla. Con 500 filas, la pantalla tarda segundos en abrir.

`FlatList`

Sólo renderiza lo visible y un poco más. Es lo que viene con React Native y para la mayoría de los casos alcanza.

> Si las filas tienen todas la misma altura, `getItemLayout` le evita a `FlatList` tener que medirlas y se nota bastante. Y la `key` estable de la clase anterior también es un tema de rendimiento, no sólo de corrección.

Otros clásicos

Animaciones que trabajan en JS

Con la API `Animated`, cada cuadro se calcula en el hilo de JavaScript salvo que pongas `useNativeDriver: true`.

Redimensionar imágenes

En iOS, cambiar `width` o `height` de una `Image` la recorta y reescala cada vez. Animá con `transform: [{ scale }]`.

`onPress` que hace mucho

Si el trabajo pesado ocurre en el mismo cuadro que el efecto visual del toque, el botón se siente muerto. Envolvelo en `requestAnimationFrame`.

Texto transparente sobre imagen

En Android obliga a recomponer en cada cuadro. `renderToHardwareTextureAndroid` ayuda — con moderación, porque come memoria.

El orden correcto

1 · Reproducir
En un dispositivo real y en build de release

2 · Medir
Perf Monitor, Profiler, resaltado de re-renderizados

3 · Cambiar
Una sola cosa por vez

4 · Volver a medir
¿Mejoró? Si no, revertí

> Optimizar sin medir agrega complejidad y casi nunca velocidad. Un `useMemo` puesto «por las dudas» es código que alguien va a tener que entender después, a cambio de nada.

UNIDAD 2 · DOCUMENTACIÓN

Documentación

Todo lo de esta clase sale de acá. Está en inglés y se mantiene al día.

Depuración

Depuración, conceptos básicos Menú de desarrollo, LogBox, Perf Monitor	`reactnative.dev/docs/debugging`
React Native DevTools Guía completa de todos los paneles	`reactnative.dev/docs/react-native-devtools`
Herramientas de depuración en Expo El panel Expo Network, VS Code, Rozenite	`docs.expo.dev/debugging/tools`
Errores y advertencias	`docs.expo.dev/debugging/errors-and-warnings`
Chrome DevTools La base sobre la que está construido todo	`developer.chrome.com/docs/devtools`

Rendimiento

Panorama de rendimiento Cuadros, hilos y causas frecuentes	`reactnative.dev/docs/performance`
Optimizar `FlatList`	`reactnative.dev/docs/optimizing-flatlist-configuration`
Perfilado	`reactnative.dev/docs/profiling`
React Profiler	`react.dev/reference/dev-tools/react-performance-tracks`

> Las páginas de `reactnative.dev` están versionadas: verificá que arriba a la izquierda diga **0.86**, que es la versión de nuestro SDK.

UNIDAD 2

Para llevarse

- `j` en la terminal abre las DevTools. Es todo lo que hay que recordar para empezar.
- Un **breakpoint** te muestra más que veinte `console.log`.
- **Medir antes de optimizar**, y siempre en release.
- Lo que sigue: **testing** — cómo evitar que el error vuelva.